

Code Layout, Readability & Reusability

A maintainable structure for the Comic Craft project

Project planning document • Phase 5: Project Development

Purpose

This document turns the project development phase into a practical, reviewable guide for the Comic Craft application. It is written for student developers and project reviewers.

1. Why structure matters

A predictable code layout helps every contributor find the right file, understand how a feature works, and make changes without breaking unrelated parts. Readability reduces review time; reusable components keep common behavior consistent.

Guiding rule: organize code by responsibility, give names that explain intent, and reuse only where the behavior is genuinely shared.

2. Suggested project layout

Area	Responsibility
src / app	Application entry point, routing, and global configuration
components	Reusable interface pieces such as comic cards, buttons, and navigation
features	Feature-specific screens and logic: library, reader, creator, profile
services	API calls, authentication, storage, and external integrations
hooks / utils	Shared stateful hooks and small pure helper functions
assets	Images, icons, fonts, and other static resources
tests	Checks grouped to mirror the source features
docs	Setup notes, decisions, and project documentation

Keep the exact folder names aligned with the framework already selected for the project. Avoid creating folders that contain only one unclear file or moving files merely to make the tree look larger.

3. Readability conventions

Names and functions

- Use descriptive names such as `ComicCard`, `fetchComicById`, and `isPublished`.
- Keep functions focused on one job; split long flows into named steps.
- Prefer explicit conditions and early returns when they make the path easier to follow.

Formatting and comments

- Apply one formatter and lint configuration consistently across the team.
- Use comments to explain a decision or constraint, not to repeat the code.
- Keep related state and logic close to the feature that owns them.

4. Reusable components and safe reuse

A component is a good candidate for reuse when it appears in multiple places and has the same meaning and behavior. Give it a small, clear interface and sensible defaults. Avoid adding many optional switches to one component when distinct components would be easier to understand.

- Extract repeated UI after the pattern is clear.
- Keep data fetching separate from presentational components where practical.
- Share design tokens for colors, spacing, and typography.
- Document unusual props and important side effects.

5. Team workflow and review checklist

- Agree on naming, formatting, and folder ownership before parallel implementation.
- Make small commits that describe one change.
- Review for clarity, duplication, error handling, and accessibility before merging.
- Update documentation when setup, architecture, or user flows change.

Review question	Ready when...
Findability	A teammate can identify where the feature belongs.
Clarity	Names and control flow explain the intent.
Reuse	Shared code has a real second use and a simple interface.
Safety	Errors and loading/empty states are considered.
Consistency	Formatting and project conventions are followed.

6. Definition of done

The code is complete when the feature is placed in the agreed structure, readable without extra explanation, reuses established patterns appropriately, and has updated supporting documentation. The team records any exception to the conventions so future contributors understand it.